MIT 8103 Advanced Database Systems: Continuous Assessment Portfolio

SpatialHub: Designing the Database for a Location-Based AR Platform

Awodi Abdulmujeeb Ayomide | Matric No. 2026/A/MIT/0184 | Student ID 301892054 | Master of Information Technology, 2026/2027 First Semester

GitHub repository: https://github.com/Platinum04/advanced-database-portfolio

1. The Organisation I Chose

SpatialHub is a simulated company. It runs a WebXR and mobile augmented reality platform that pins 3D models to real places in Lagos, Abuja and London using spatial anchors. Creators upload models and build experiences such as heritage trails and museum guides, and visitors see the content appear when they walk near an anchor on their phone, in a headset or in a WebXR browser. The platform also sells tickets to live AR events. The people using the database are visitors, creators, admins and the AR app itself. It stores users, cities, venues, 3D assets, experiences and tags, anchor locations, events, tickets, payments, AR sessions, interactions and device telemetry. The main jobs it has to do are finding anchors near a user, loading a full scene, selling a limited number of tickets without overselling, saving payments, and reporting on engagement and device performance. Because it combines location data, payments and a steady stream of device data, it gave me material for every part of the course.

2. The Five Portfolio Activities

Portfolio 1: Database design and modelling. I designed 11 tables in PostgreSQL 16 with PostGIS and drew the ER diagram in Mermaid. Every table has an identity column as its primary key, with natural keys like email protected by UNIQUE constraints. The junction table for tags has a composite key. There are 17 foreign keys and CHECK constraints for things like the anchor heading (0 to 360), events ending after they start, and tickets sold never going above capacity. Anchor locations use the GEOGRAPHY type so distances come out in metres. I normalised a flat "anchor spreadsheet" step by step up to BCNF, and kept one deliberate exception (tickets_sold on the event) to make checkout simpler. When I tried 8 inserts that should be impossible, the database rejected all of them.

Portfolio 2: Query processing and optimisation. I generated 1.4 million rows (200k anchors, 200k sessions and 1 million interactions) and ran four real queries with EXPLAIN (ANALYZE, BUFFERS) before and after adding indexes. Each index was chosen for its query: GiST for the spatial search, a two column B-tree in the same order as the sort, BRIN for the interaction log, and an index on a foreign key PostgreSQL hadn't indexed for me. I also tried a materialised view for the trending leaderboard.

Query (warm cache, JIT off)	Before: plan and time	After: plan and time	Faster by
Q1 anchors within 300 m	Full parallel scan, 168.2 ms	GiST index, 7.4 ms	about 23x
Q2 user's latest 20 sessions	Full scan plus sort, 11.3 ms	Two column B-tree, 0.12 ms	about 98x
Q3 one day of interactions	Full scan of 1M rows, 139.6 ms	BRIN (24 kB vs 21 MB for a B-tree), 8.1 ms	about 17x
Q4 engagement for one anchor	Full scan of 1M rows, 36.4 ms	Foreign key index, 0.08 ms	about 470x
Trending leaderboard	Live total, 168.2 ms	Materialised view, 0.08 ms	about 2,000x

Portfolio 3: Transactions and concurrency. The SQL script shows a normal purchase being committed, a declined payment being rolled back, a savepoint that undoes only part of a transaction, and a failed CHECK cancelling a whole transaction. Then I wrote a Python script that opens two real connections at the same moment to recreate two people buying the last ticket. With the simple read-then-write code at READ COMMITTED, both purchases went through and the event ended up with 3 tickets for 2 seats. That's a lost update. I fixed it three ways: a row lock with SELECT ... FOR UPDATE (the second buyer waited about half a second, then saw it was sold out), a single conditional UPDATE (the one I'd use), and SERIALIZABLE isolation, where the second buyer got error 40001 and retried. The same script shows a non-repeatable read under READ COMMITTED going away under REPEATABLE READ, and PostgreSQL breaking a deadlock with error 40P01.

Portfolio 4: NoSQL. I copied the relational data into MongoDB 8. Each experience became one document with its anchors and 3D model details inside it, so the app can load a whole scene in one read instead of joining 5 tables. I also loaded 200,008 anchors as GeoJSON points with a 2dsphere index, and the nearby search only looked at 36 of them and took 1 ms. Headset and phone telemetry went into a time series collection, where each device only stores the fields it actually has (eye tracking and controllers for headsets, light level for phones). An aggregation showed WebXR in the browser was the weakest platform at 42 fps on average. A validation rule rejected a document with an invalid status.

Portfolio 5: Distributed and cloud. I set up PostgreSQL streaming replication with a primary and a standby. The replica stayed 0 bytes behind, refused writes, worked in synchronous mode, and after I crashed the primary and promoted the replica, nothing was lost and new writes were accepted. Then I built a MongoDB sharded cluster with two shards and used zones on {region, _id}, so Lagos and Abuja anchors (133,333) stayed on the Africa shard and London anchors (66,667) on the Europe shard. That helps with speed and with the NDPA data rules. Queries that included the region only went to one shard, and queries without it had to ask both. When I shut down one shard, the other region kept working, which showed me

why every shard needs its own replica set. I finished with a CAP discussion (payments favour consistency, scenes and telemetry favour availability) and a cloud plan using RDS Multi-AZ and an Atlas Global Cluster.

3. Evidence

Each portfolio folder in the repository has the scripts, a README explaining what I did and what I found, a results folder with the real output from running everything, and a screenshots folder. There are 18 screenshots in total, including the query plans before and after indexing, the oversold race, the failover and the shard distribution.

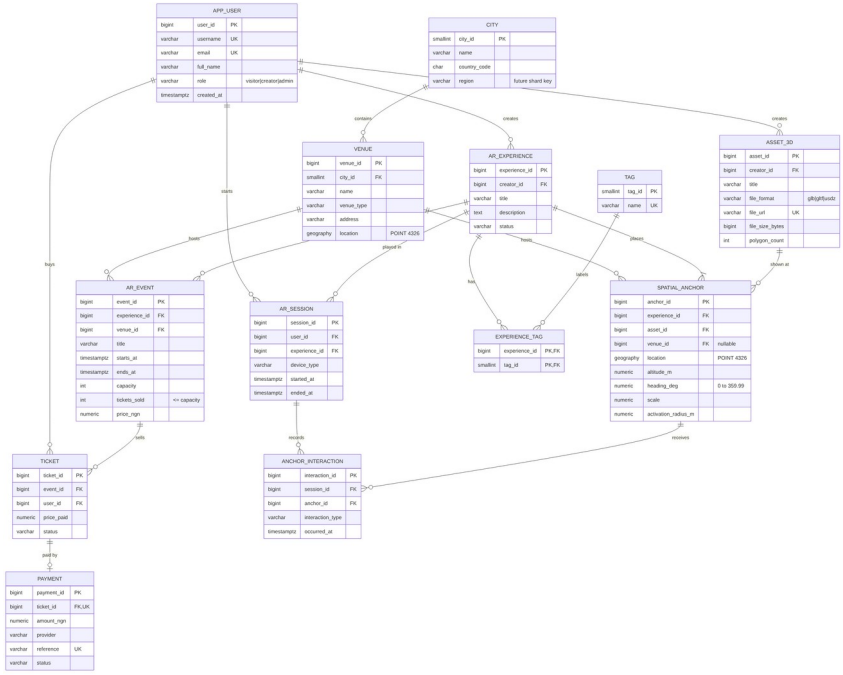


Figure 1. SpatialHub ER diagram (Portfolio 1)

4. Reflection

- **Design choices.** Storing locations as PostGIS geography from the start made a big difference later. It's what made the index comparison in Portfolio 2 interesting, and it moved straight across to GeoJSON in MongoDB. I kept tickets and payments in PostgreSQL and moved scenes and telemetry to MongoDB, because each one suits a different kind of data.
- **Problems I ran into.** First, my early benchmark showed Q1 taking 489 ms, but when I read the whole plan, 475 ms of it was JIT compilation and not the actual scan. I switched JIT off for a fair test and wrote up what happened. Second, my first sharding run put all the data on one shard. I had added the zones after sharding the collection, and the balancer hadn't moved the chunks yet. Setting up the zones before sharding the empty collection fixed it. Third, generating the big data set failed on foreign keys, because my earlier failed test inserts had used up some ID numbers. I changed the generator to only pick IDs that actually exist.
- **What I learned.** Constraints protect single rows, not business rules. The oversold race got past every CHECK I had, so concurrency has to be part of the design, and the conditional UPDATE is the simplest safe way to do it. Every index is a trade-off (BRIN is 900 times smaller but relies on rows staying in order). And in a sharded database, the shard key decides whether a query touches one server or all of them.

5. AI Use Declaration

Tool: Claude, used in the browser. **What I used it for:** drafting the SQL, Python and shell scripts, and generating the synthetic test data. **How I checked the work:** I ran all the scripts against PostgreSQL 16 with PostGIS and MongoDB 8, and the output in each results folder is what they actually printed. I checked the results against what I expected and fixed the three problems described in my reflection. I've gone through the design, the results and the explanations, and I can explain any part of the work.

6. References

Elmasri, R. and Navathe, S.B. (2016) Fundamentals of Database Systems. 7th edn. Pearson. | Kleppmann, M. (2017) Designing Data-Intensive Applications. O'Reilly. | MongoDB Inc. (2025) MongoDB Manual. https://www.mongodb.com/docs/manual/ | Nigeria Data Protection Commission (2023) Nigeria Data Protection Act 2023. | PostGIS Project (2025) PostGIS 3 Manual. https://postgis.net/documentation/ | PostgreSQL Global Development Group (2025) PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/ | Miva Open University (2026) MIT 8103 Advanced Database Systems, Study Sessions 1 to 6.